

University of Calcutta

Paper Name: Deep Learning

Paper Code: CSE1104

Assignment 1

Perceptron Learning and Decision Boundary Dynamics

Name: Rahul Biswas

Roll No.: 16

Department of Computer Science and Engineering

Contents

1	Introduction to Perceptron	2
1.1	Mathematical Fundamentals	2
2	Task 1: Realization of AND gate using perceptron	3
2.1	Introduction to the AND Gate	3
2.2	AND Gate using perceptron	3
2.2.1	Epoch 1	3
2.2.2	Epoch 2	3
2.2.3	Epoch 3	4
2.2.4	Epoch 4	4
2.2.5	Epoch 5	4
2.3	Geometric Interpretation	5
2.4	Conclusion	6
3	Task 2: Realization of OR Gate Using Perceptron	7
3.1	Introduction to the OR Gate	7
3.2	OR Gate Using Perceptron	7
3.2.1	Epoch 1	7
3.2.2	Epoch 2	8
3.3	Geometric Interpretation	8
3.4	Conclusion	9
4	Task 3: Realization of XOR Gate Using Perceptron	10
4.1	Introduction to the XOR Gate	10
4.2	XOR Gate Using Perceptron	10
4.2.1	Epoch 1	10
4.2.2	Epoch 2	11
4.2.3	Epoch 3	11
4.2.4	Epoch 4	11
4.2.5	Epoch 5	11
4.2.6	Epoch 6	11
4.2.7	Epoch 7	12
4.2.8	Epoch 8	12
4.2.9	Epoch 9	12
4.2.10	Epoch 10	12
4.3	Geometric Interpretation	13
4.4	Conclusion	15
5	Task 4: Realization of Half Adder	16
5.1	Introduction to the Half Adder	16
5.2	Half Adder Using Perceptrons	16
5.3	Perceptron Representation of the Carry Output	17
5.4	Perceptron Representation of the Sum Output	17
5.5	Conclusion	17
6	Reference links	18

1 Introduction to Perceptron

The perceptron was first introduced by Frank Rosenblatt in 1958 as a simple computational model inspired by the biological neuron.

1.1 Mathematical Fundamentals

Consider a perceptron with n input features:

$$x_1, x_2, x_3, \dots, x_n$$

Each input is associated with a corresponding weight:

$$w_1, w_2, w_3, \dots, w_n$$

The weighted sum of the inputs is calculated as:

$$z = \sum_{i=1}^n w_i x_i + b \quad (1)$$

where b represents the bias term.

The output of the perceptron is determined by applying an activation function to the weighted sum. The commonly used step activation function is defined as:

$$\hat{y} = f(z) = \begin{cases} 1, & \text{if } z \geq 0, \\ 0, & \text{if } z < 0. \end{cases} \quad (2)$$

The error term can be defined as:

$$E = (\hat{y} - y) \quad (3)$$

After computing the error term E , the learnable parameters $(w_1, w_2, \dots, w_n, b)$ are updated to linearly separate the input features.

The learnable parameters are updated using:

$$w'_i = w_i + \lambda * E * x_i \quad \forall i \in \{1, 2, \dots, n\} \quad (4)$$

$$b_1 = b_1 + \lambda * E \quad (5)$$

where λ represents the learning rate.

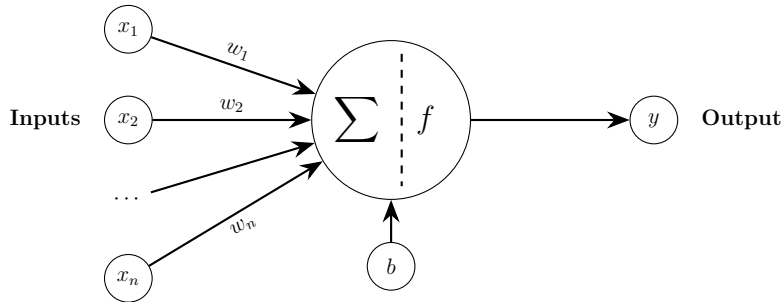


Figure 1: Structure of a Single Neuron Perceptron

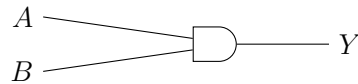
2 Task 1: Realization of AND gate using perceptron

2.1 Introduction to the AND Gate

Truth Table

A	B	$Y = A \cdot B$
0	0	0
0	1	0
1	0	0
1	1	1

Logical Diagram



The AND gate produces an output of 1 only when both inputs A and B are equal to 1. For all other input combinations, the output is 0. Mathematically, the AND operation can be expressed as:

$$Y = A \cdot B \quad (6)$$

2.2 AND Gate using perceptron

For the AND gate, the inputs to the perceptron are:

$$\mathbf{x} = (x_1, x_2) = (A, B) = \{0, 0\}, \{0, 1\}, \{1, 0\}, \{1, 1\}$$

The corresponding expected outputs are:

$$y = 0, 0, 0, 1$$

Assume that the learnable parameters are initialized as follows:

$$w_1 = 1 \quad w_2 = 1 \quad b = 0 \quad (7)$$

When the perceptron is trained using the above settings, the following results are obtained for each epoch:

2.2.1 Epoch 1

x_1	x_2	y	z	$\hat{y}$	E	w_1	w_2	b	w'_1	w'_2	b'
0	0	0	0.0	1	-1	1.0	1.0	0.0	1.0	1.0	-1.0
0	1	0	0.0	1	-1	1.0	1.0	-1.0	1.0	0.0	-2.0
1	0	0	-1.0	0	0	1.0	0.0	-2.0	1.0	0.0	-2.0
1	1	1	-1.0	0	1	1.0	0.0	-2.0	2.0	1.0	-1.0

2.2.2 Epoch 2

x_1	x_2	y	z	$\hat{y}$	E	w_1	w_2	b	w'_1	w'_2	b'
0	0	0	-1.0	0	0	2.0	1.0	-1.0	2.0	1.0	-1.0
0	1	0	0.0	1	-1	2.0	1.0	-1.0	2.0	0.0	-2.0
1	0	0	0.0	1	-1	2.0	0.0	-2.0	1.0	0.0	-3.0
1	1	1	-2.0	0	1	2.0	0.0	-3.0	2.0	1.0	-2.0

2.2.3 Epoch 3

x_1	x_2	y	z	$\hat{y}$	E	w_1	w_2	b	w'_1	w'_2	b'
0	0	0	-2.0	0	0	2.0	1.0	-2.0	2.0	1.0	-2.0
0	1	0	-1.0	0	0	2.0	1.0	-2.0	2.0	1.0	-2.0
1	0	0	0.0	1	-1	2.0	1.0	-2.0	1.0	1.0	-3.0
1	1	1	-1.0	0	1	1.0	1.0	-3.0	2.0	2.0	-2.0

2.2.4 Epoch 4

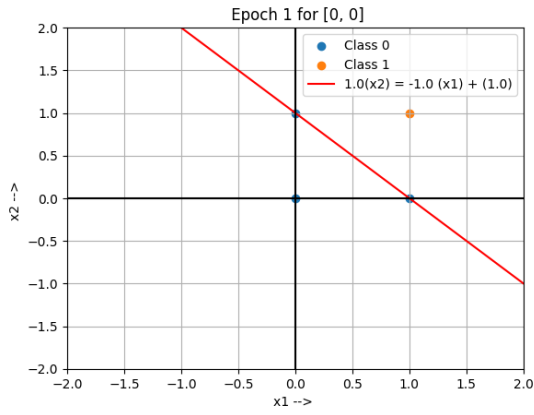
x_1	x_2	y	z	$\hat{y}$	E	w_1	w_2	b	w'_1	w'_2	b'
0	0	0	-2.0	0	0	2.0	2.0	-2.0	2.0	2.0	-2.0
0	1	0	0.0	1	-1	2.0	2.0	-2.0	2.0	1.0	-3.0
1	0	0	-1.0	0	0	2.0	1.0	-3.0	2.0	1.0	-3.0
1	1	1	0.0	1	0	2.0	1.0	-3.0	2.0	1.0	-3.0

2.2.5 Epoch 5

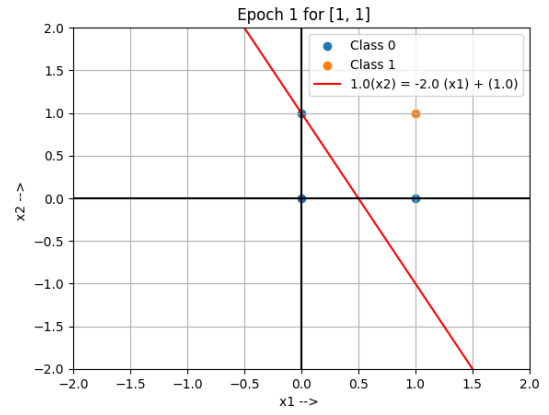
x_1	x_2	y	z	$\hat{y}$	E	w_1	w_2	b	w'_1	w'_2	b'
0	0	0	-3.0	0	0	2.0	1.0	-3.0	2.0	1.0	-3.0
0	1	0	-2.0	0	0	2.0	1.0	-3.0	2.0	1.0	-3.0
1	0	0	-1.0	0	0	2.0	1.0	-3.0	2.0	1.0	-3.0
1	1	1	0.0	1	0	2.0	1.0	-3.0	2.0	1.0	-3.0

2.3 Geometric Interpretation

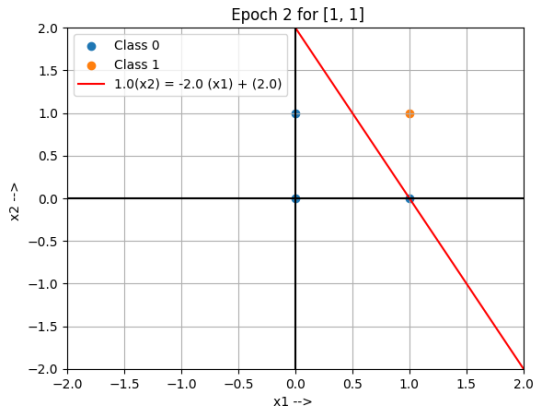
The **geometric interpretations** corresponding to different values of the learnable parameters are illustrated below.



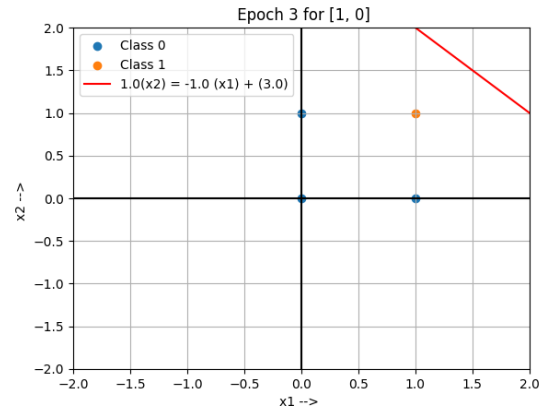
(a) Plot 1



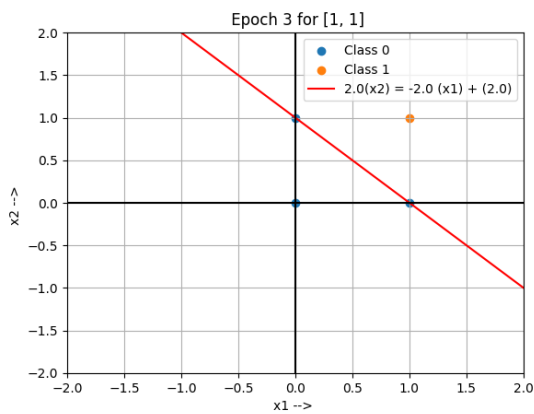
(b) Plot 2



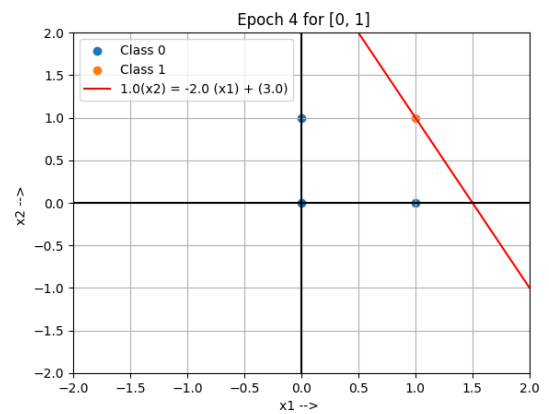
(c) Plot 3



(d) Plot 4



(e) Plot 5



(f) Plot 6

Figure 2: Decision Boundary Dynamics Across Different Epochs

2.4 Conclusion

In conclusion, when the perceptron is initialized with the learnable parameters specified in Equation 7, it converges after the fifth epoch. The resulting trained parameters are:

$$w_1 = 2 \quad w_2 = 1 \quad b = -3 \quad (8)$$

Furthermore, the decision boundary formed by these parameters demonstrates that the hyperplane linearly separates the two classes. Therefore, a perceptron with these trained parameters can successfully realize the AND gate.

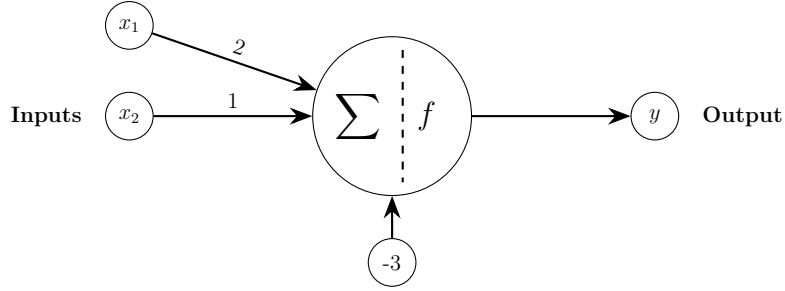


Figure 3: AND Gate using perceptron

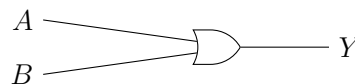
3 Task 2: Realization of OR Gate Using Perceptron

3.1 Introduction to the OR Gate

Truth Table

A	B	$Y = A + B$
0	0	0
0	1	1
1	0	1
1	1	1

Logical Diagram



The OR gate produces an output of 1 when at least one of the inputs A or B is equal to 1. The output is 0 only when both inputs are equal to 0. Mathematically, the OR operation can be expressed as:

$$Y = A + B \quad (9)$$

3.2 OR Gate Using Perceptron

For the OR gate, the inputs to the perceptron are:

$$\mathbf{x} = (x_1, x_2) = (A, B) = \{0, 0\}, \{0, 1\}, \{1, 0\}, \{1, 1\}$$

The corresponding expected outputs are:

$$y = \{0, 1, 1, 1\}$$

Assume that the learnable parameters are initialized as follows:

$$w_1 = 1 \quad w_2 = 1 \quad b = 0 \quad (10)$$

When the perceptron is trained using the above settings, the following results are obtained for each epoch:

3.2.1 Epoch 1

x_1	x_2	y	z	$\hat{y}$	E	w_1	w_2	b	w'_1	w'_2	b'
0	0	0	0.0	1	-1	1.0	1.0	0.0	1.0	1.0	-1.0
0	1	1	0.0	1	0	1.0	1.0	-1.0	1.0	1.0	-1.0
1	0	1	0.0	1	0	1.0	1.0	-1.0	1.0	1.0	-1.0
1	1	1	1.0	1	0	1.0	1.0	-1.0	1.0	1.0	-1.0

3.2.2 Epoch 2

x_1	x_2	y	z	$\hat{y}$	E	w_1	w_2	b	w'_1	w'_2	b'
0	0	0	-1.0	0	0	1.0	1.0	-1.0	1.0	1.0	-1.0
0	1	1	0.0	1	0	1.0	1.0	-1.0	1.0	1.0	-1.0
1	0	1	0.0	1	0	1.0	1.0	-1.0	1.0	1.0	-1.0
1	1	1	1.0	1	0	1.0	1.0	-1.0	1.0	1.0	-1.0

3.3 Geometric Interpretation

The geometric interpretation of the final learnable parameters is illustrated below.

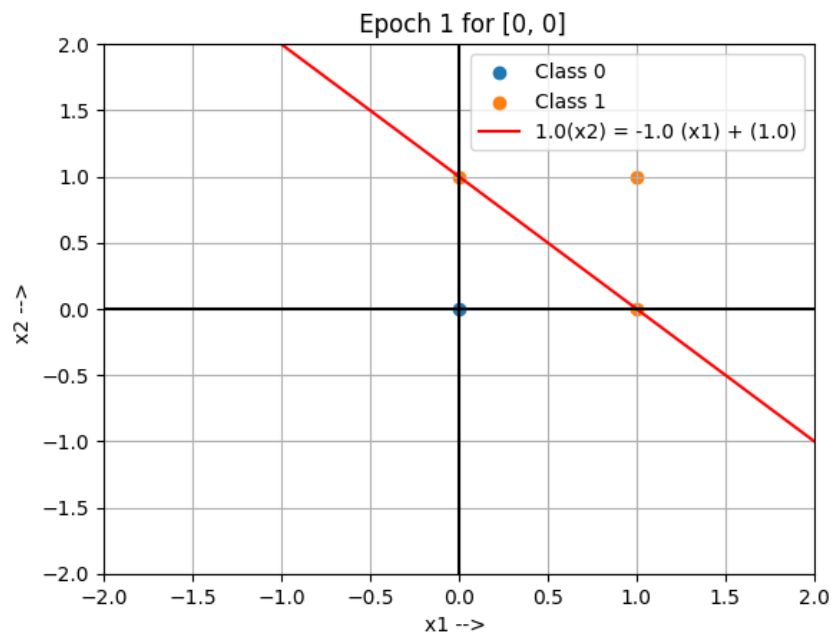


Figure 4: Decision Boundary for the OR Gate

3.4 Conclusion

From the training results, it can be observed that the perceptron converges after the 2nd epoch. The resulting trained parameters are:

$$w_1 = 1 \quad w_2 = 1 \quad b = -1 \quad (11)$$

The decision boundary formed by these parameters linearly separates the two classes of the OR gate. Therefore, the trained perceptron can successfully realize the OR gate.

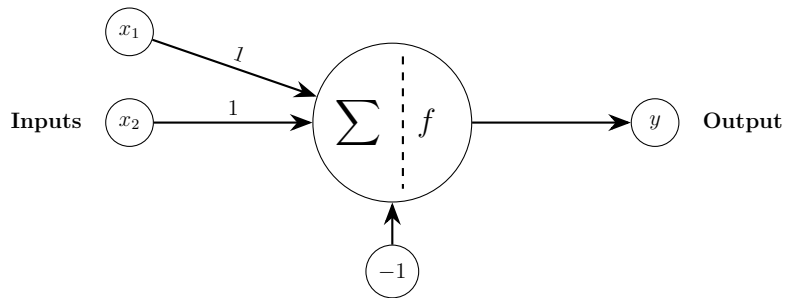


Figure 5: OR Gate Using Perceptron

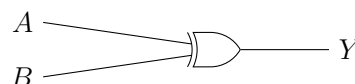
4 Task 3: Realization of XOR Gate Using Perceptron

4.1 Introduction to the XOR Gate

Truth Table

A	B	$Y = A \oplus B$
0	0	0
0	1	1
1	0	1
1	1	0

Logical Diagram



The XOR gate produces an output of 1 when the two inputs are different. When both inputs are equal, the output is 0. Mathematically, the XOR operation can be expressed as:

$$Y = A \oplus B \quad (12)$$

4.2 XOR Gate Using Perceptron

For the XOR gate, the inputs to the perceptron are:

$$\mathbf{x} = (x_1, x_2) = (A, B) = \{0, 0\}, \{0, 1\}, \{1, 0\}, \{1, 1\}$$

The corresponding expected outputs are:

$$y = \{0, 1, 1, 0\}$$

Assume that the learnable parameters are initialized as follows:

$$w_1 = 1 \quad w_2 = 1 \quad b = 0 \quad (13)$$

When the perceptron is trained using the above settings, the following results are obtained for each epoch:

4.2.1 Epoch 1

x_1	x_2	y	z	$\hat{y}$	E	w_1	w_2	b	w'_1	w'_2	b'
0	0	0	0.0	1	-1	1.0	1.0	0.0	1.0	1.0	-1.0
0	1	1	0.0	1	0	1.0	1.0	-1.0	1.0	1.0	-1.0
1	0	1	0.0	1	0	1.0	1.0	-1.0	1.0	1.0	-1.0
1	1	0	1.0	1	-1	1.0	1.0	-1.0	0.0	0.0	-2.0

4.2.2 Epoch 2

x_1	x_2	y	z	$\hat{y}$	E	w_1	w_2	b	w'_1	w'_2	b'
0	0	0	-2.0	0	0	0.0	0.0	-2.0	0.0	0.0	-2.0
0	1	1	-2.0	0	1	0.0	0.0	-2.0	0.0	1.0	-1.0
1	0	1	-1.0	0	1	0.0	1.0	-1.0	1.0	1.0	0.0
1	1	0	2.0	1	-1	1.0	1.0	0.0	0.0	0.0	-1.0

4.2.3 Epoch 3

x_1	x_2	y	z	$\hat{y}$	E	w_1	w_2	b	w'_1	w'_2	b'
0	0	0	-1.0	0	0	0.0	0.0	-1.0	0.0	0.0	-1.0
0	1	1	-1.0	0	1	0.0	0.0	-1.0	0.0	1.0	0.0
1	0	1	0.0	1	0	0.0	1.0	0.0	0.0	1.0	0.0
1	1	0	1.0	1	-1	0.0	1.0	0.0	-1.0	0.0	-1.0

4.2.4 Epoch 4

x_1	x_2	y	z	$\hat{y}$	E	w_1	w_2	b	w'_1	w'_2	b'
0	0	0	-1.0	0	0	-1.0	0.0	-1.0	-1.0	0.0	-1.0
0	1	1	-1.0	0	1	-1.0	0.0	-1.0	-1.0	1.0	0.0
1	0	1	-1.0	0	1	-1.0	1.0	0.0	0.0	1.0	1.0
1	1	0	2.0	1	-1	0.0	1.0	1.0	-1.0	0.0	0.0

4.2.5 Epoch 5

x_1	x_2	y	z	$\hat{y}$	E	w_1	w_2	b	w'_1	w'_2	b'
0	0	0	0.0	1	-1	-1.0	0.0	0.0	-1.0	0.0	-1.0
0	1	1	-1.0	0	1	-1.0	0.0	-1.0	-1.0	1.0	0.0
1	0	1	-1.0	0	1	-1.0	1.0	0.0	0.0	1.0	1.0
1	1	0	2.0	1	-1	0.0	1.0	1.0	-1.0	0.0	0.0

4.2.6 Epoch 6

x_1	x_2	y	z	$\hat{y}$	E	w_1	w_2	b	w'_1	w'_2	b'
0	0	0	0.0	1	-1	-1.0	0.0	0.0	-1.0	0.0	-1.0
0	1	1	-1.0	0	1	-1.0	0.0	-1.0	-1.0	1.0	0.0
1	0	1	-1.0	0	1	-1.0	1.0	0.0	0.0	1.0	1.0
1	1	0	2.0	1	-1	0.0	1.0	1.0	-1.0	0.0	0.0

4.2.7 Epoch 7

x_1	x_2	y	z	$\hat{y}$	E	w_1	w_2	b	w'_1	w'_2	b'
0	0	0	0.0	1	-1	-1.0	0.0	0.0	-1.0	0.0	-1.0
0	1	1	-1.0	0	1	-1.0	0.0	-1.0	-1.0	1.0	0.0
1	0	1	-1.0	0	1	-1.0	1.0	0.0	0.0	1.0	1.0
1	1	0	2.0	1	-1	0.0	1.0	1.0	-1.0	0.0	0.0

4.2.8 Epoch 8

x_1	x_2	y	z	$\hat{y}$	E	w_1	w_2	b	w'_1	w'_2	b'
0	0	0	0.0	1	-1	-1.0	0.0	0.0	-1.0	0.0	-1.0
0	1	1	-1.0	0	1	-1.0	0.0	-1.0	-1.0	1.0	0.0
1	0	1	-1.0	0	1	-1.0	1.0	0.0	0.0	1.0	1.0
1	1	0	2.0	1	-1	0.0	1.0	1.0	-1.0	0.0	0.0

4.2.9 Epoch 9

x_1	x_2	y	z	$\hat{y}$	E	w_1	w_2	b	w'_1	w'_2	b'
0	0	0	0.0	1	-1	-1.0	0.0	0.0	-1.0	0.0	-1.0
0	1	1	-1.0	0	1	-1.0	0.0	-1.0	-1.0	1.0	0.0
1	0	1	-1.0	0	1	-1.0	1.0	0.0	0.0	1.0	1.0
1	1	0	2.0	1	-1	0.0	1.0	1.0	-1.0	0.0	0.0

4.2.10 Epoch 10

x_1	x_2	y	z	$\hat{y}$	E	w_1	w_2	b	w'_1	w'_2	b'
0	0	0	0.0	1	-1	-1.0	0.0	0.0	-1.0	0.0	-1.0
0	1	1	-1.0	0	1	-1.0	0.0	-1.0	-1.0	1.0	0.0
1	0	1	-1.0	0	1	-1.0	1.0	0.0	0.0	1.0	1.0
1	1	0	2.0	1	-1	0.0	1.0	1.0	-1.0	0.0	0.0

4.3 Geometric Interpretation

The geometric interpretations corresponding to the different values of the learnable parameters during the training process are illustrated below.

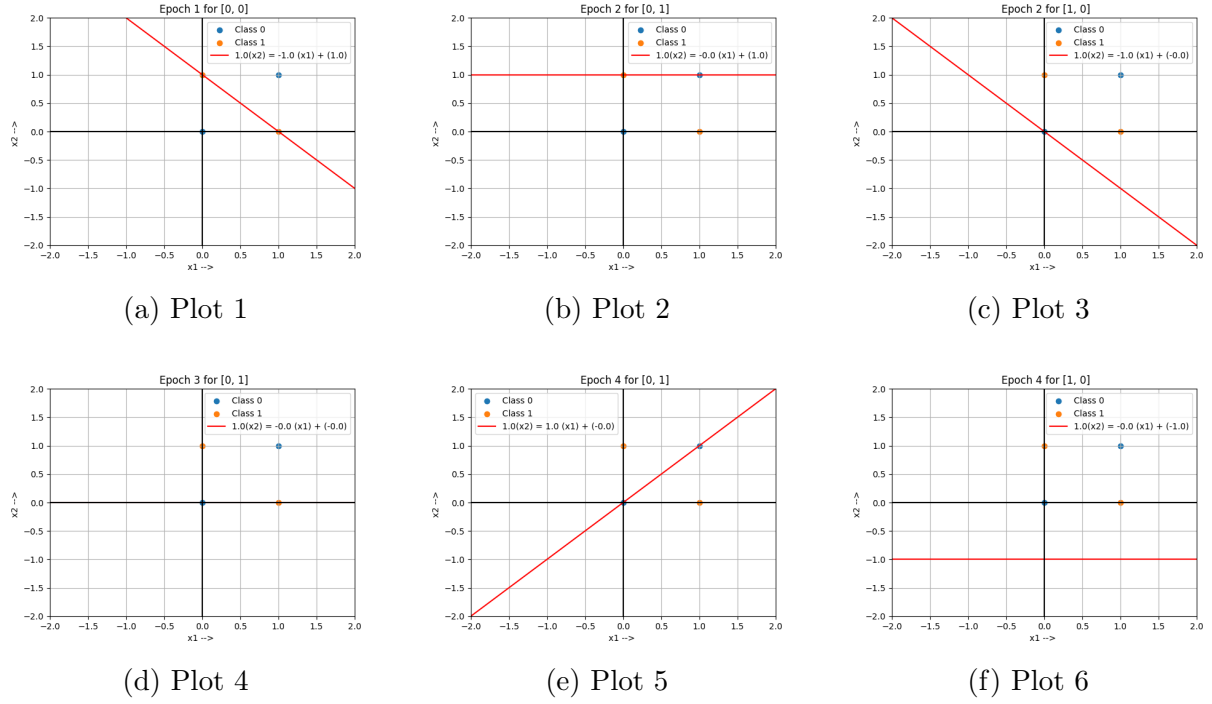
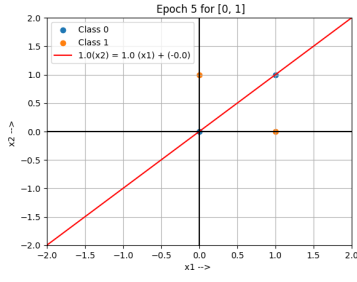
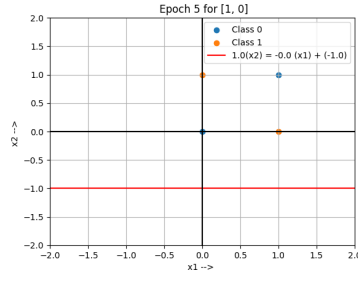


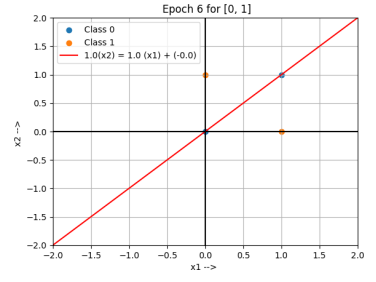
Figure 6: Decision Boundary Dynamics for the XOR Gate



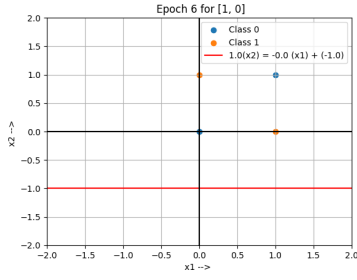
(a) Plot 7



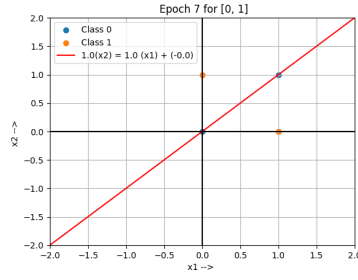
(b) Plot 8



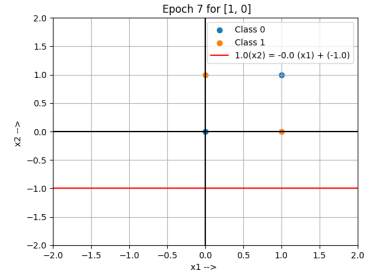
(c) Plot 9



(d) Plot 10

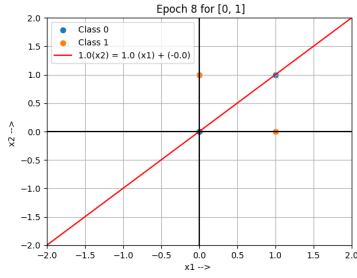


(e) Plot 11

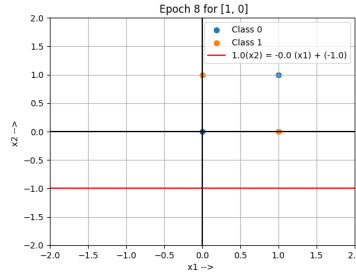


(f) Plot 12

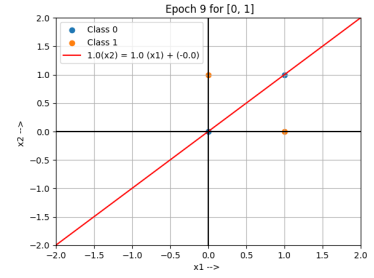
Figure 7: Decision Boundary Dynamics for the XOR Gate



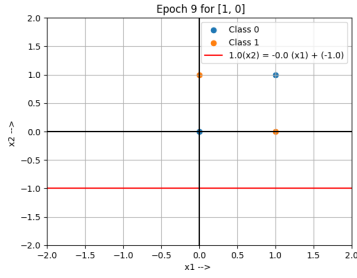
(a) Plot 13



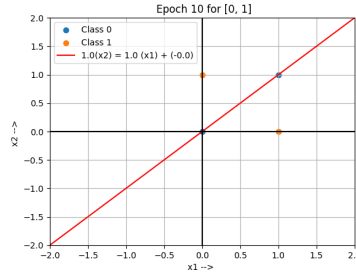
(b) Plot 14



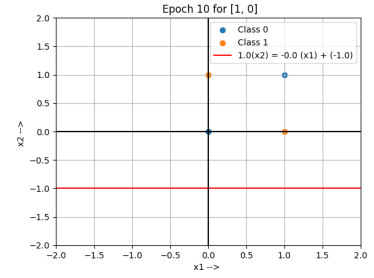
(c) Plot 15



(d) Plot 16



(e) Plot 17



(f) Plot 18

Figure 8: Decision Boundary Dynamics for the XOR Gate

4.4 Conclusion

From the training results, it can be observed that the perceptron does not converge to a set of parameters that correctly classifies all the input patterns of the XOR gate and we need to hard stop after 10 epochs.

The reason for this behavior is that the XOR problem is not linearly separable. The input combinations that produce an output of 1, namely (0, 1) and (1, 0), cannot be separated from the input combinations that produce an output of 0, namely (0, 0) and (1, 1), using a single linear decision boundary. This problem is known as XOR problem. This problem was first introduced in the book named "Perceptrons" in the year 1969.

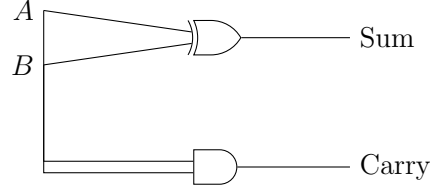
5 Task 4: Realization of Half Adder

5.1 Introduction to the Half Adder

Truth Table

A	B	Sum	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Logical Diagram



A half adder is a digital logic circuit used to add two single-bit binary numbers. It has two inputs, A and B , and produces two outputs: the *Sum* and the *Carry*. The Sum output is obtained using an XOR operation, while the Carry output is obtained using an AND operation.

The mathematical expressions for the outputs are:

$$S = A \oplus B \quad (14)$$

$$C = A \cdot B \quad (15)$$

5.2 Half Adder Using Perceptrons

A half adder consists of two logical operations: XOR for generating the Sum output and AND for generating the Carry output.

The inputs to the half adder are:

$$\mathbf{x} = (x_1, x_2) = (A, B) = \{0, 0\}, \{0, 1\}, \{1, 0\}, \{1, 1\}$$

The corresponding Sum outputs are:

$$S = \{0, 1, 1, 0\}$$

The corresponding Carry outputs are:

$$C = \{0, 0, 0, 1\}$$

The Carry output can be directly realized using a single perceptron because the AND operation is linearly separable. However, the Sum output corresponds to the XOR operation, which is not linearly separable and therefore cannot be realized using a single perceptron.

Consequently, a multilayer perceptron architecture is required to realize the complete half adder.

5.3 Perceptron Representation of the Carry Output

For the Carry output, the learnable parameters are initialized and trained to realize the AND operation.

The final trained parameters are:

$$w_1 = 2 \quad w_2 = 1 \quad b = -3 \quad (16)$$

The resulting decision boundary separates the input (1, 1) from all other input combinations. Therefore, the Carry output can be successfully realized using a single perceptron.

5.4 Perceptron Representation of the Sum Output

The Sum output follows the XOR operation:

$$S = A \oplus B$$

The XOR operation produces an output of 1 for the input combinations (0, 1) and (1, 0), while producing an output of 0 for (0, 0) and (1, 1).

These two classes cannot be separated by a single linear decision boundary. Therefore, a single perceptron cannot successfully realize the Sum output.

5.5 Conclusion

In conclusion, a half adder performs the binary addition of two input bits and produces two outputs: Sum and Carry.

The Carry output is equivalent to the AND operation and can be successfully implemented using a single perceptron because the data is linearly separable.

The Sum output is equivalent to the XOR operation and cannot be implemented using a single perceptron because XOR is not linearly separable. Therefore, a multilayer perceptron is required to realize the complete half adder.

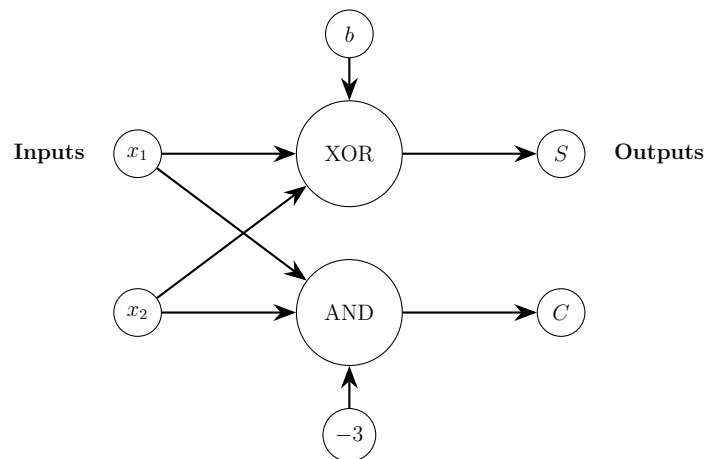


Figure 9: Half Adder Using Perceptron-Based Architecture

6 Reference links

For code and output click here: github.com